

EARLY PREDICTION OF CHRONIC KIDNEY DISEASE

1. INTRODUCTION

1.1 Overview

Chronic Kidney Disease (CKD) is a major medical problem characterized by the progressive loss of kidney function. If treated in the early stages, CKD can often be managed or cured. However, people are usually not aware that routine medical tests could contain valuable information concerning kidney diseases. Consequently, this project investigates attributes of various medical tests to distinguish which ones may contain helpful information about the disease. We use this information to build a machine learning model that predicts Chronic Kidney Disease (CKD).

1.2 Purpose

This project aims to provide an accessible and highly accurate tool for predicting CKD status. By completing this work, users will be able to:

- Understand the problem as a **classification** kind of problem.
- Know how to **preprocess and clean** clinical data using various data preprocessing techniques.
- Analyze and gain insights from the data through visualization.
- Apply different machine learning **algorithms** according to the dataset.
- Determine the **accuracy** of the model.
- Build a web application using the **Flask framework** to integrate the model.

2. LITERATURE SURVEY

2.1 Existing Problem

Chronic Kidney Disease (CKD) or chronic renal disease has become a major issue with a steady growth rate. The critical nature of the disease, where a person can only survive for a short time without kidneys, makes kidney transplants and dialysis highly demanded. The existing challenge is that models used previously often didn't give accurate results, and the

time taken to predict the outcome was lengthy. Effective methods for the **early prediction of CKD** are therefore critical.

2.2 Proposed Solution

This work proposes a workflow utilizing **Machine Learning methods** for effective CKD prediction. The solution incorporates robust data preprocessing, including handling missing values and performing attribute selection. We implement and evaluate several classifiers, including Random Forest, Gaussian Naïve Bayes, and Logistic Regression. By leveraging these techniques and splitting the data into training and testing sets, we solve the existing problem of low accuracy and slow prediction time.

3. THEORITICAL ANALYSIS

The theoretical analysis involves designing the system architecture and defining the hardware/software requirements.

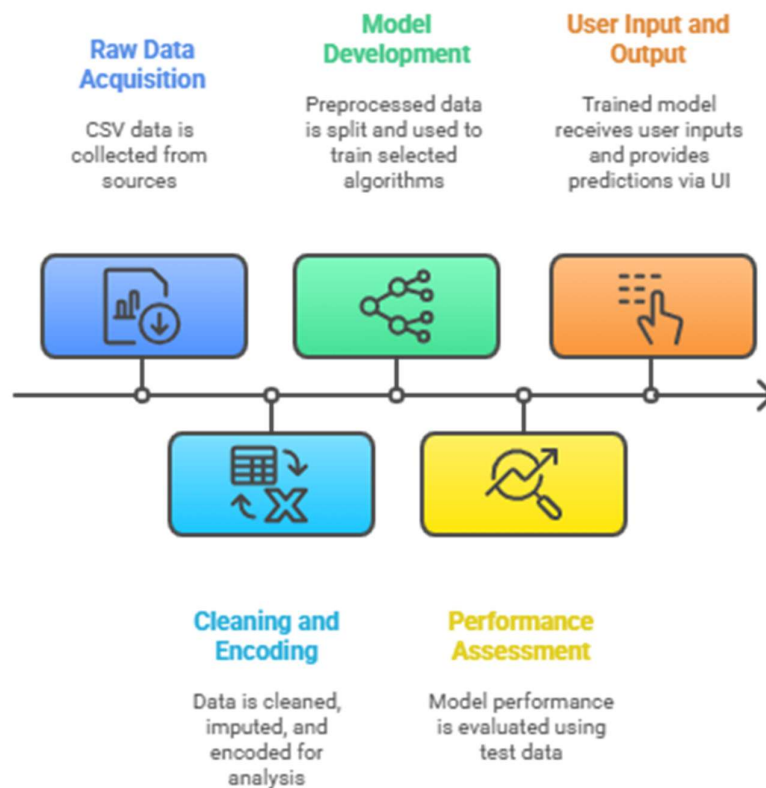
Block Diagram

The system operates based on the following flow:

- **Data Collection:** Raw data (CSV format) is collected.
- **Data Preprocessing:** Data is cleaned, imputed, and encoded.
- **Algorithm/Model Training:** The preprocessed data is split into Train/Test sets and fed into the selected algorithms for training.
- **Evaluation:** The model performance is assessed based on test data.
- **Prediction:** The trained model receives user inputs via the UI and provides a prediction.

The block diagram is as follows:

Data Science Workflow Stages



4. EXPERIMENTAL INVESTIGATIONS

4.1 Data Collection

The project uses a training dataset obtained from **Kaggle**. The dataset contains **400 entries** and **25 columns** of clinical data, serving as the most crucial aspect for enabling algorithm training.

4.2 Data Preprocessing

Data preprocessing was executed in several key activities:

1. **Imputation:** Missing values in numerical features (like *age*, *blood pressure*, etc.) were filled using the **mean** of the column. Missing values in categorical features (like *red blood cells*, *pus cell*, etc.) were filled using the **mode** (most frequent value).
2. **Cleaning Inconsistent Data:** Inconsistent entries in the data were corrected (e.g., `$\text{\`tno\`}` was replaced with 'no', `$\text{\`ckd\t\`}` was replaced with 'ckd'.

3. **Label Encoding:** Categorical variables were converted into numerical form using **LabelEncoder** from scikit-learn, as machine learning algorithms operate best with numerical data.
4. **Feature Selection:** The final feature matrix (X) was created by dropping the target variable 'classification' and features like 'pot', 'ba', 'wc', 'age', 'su', and 'ane'.
5. **Data Splitting:** The data was split into training and testing sets (75% training, 25% testing) using the **train_test_split** class from scikit-learn.

4.3 Model Building and Training

We initialized, trained, and evaluated several classifiers:

1. **Random Forest Classifier.**
2. **Gaussian Naïve Bayes.**
3. **Logistic Regression.**

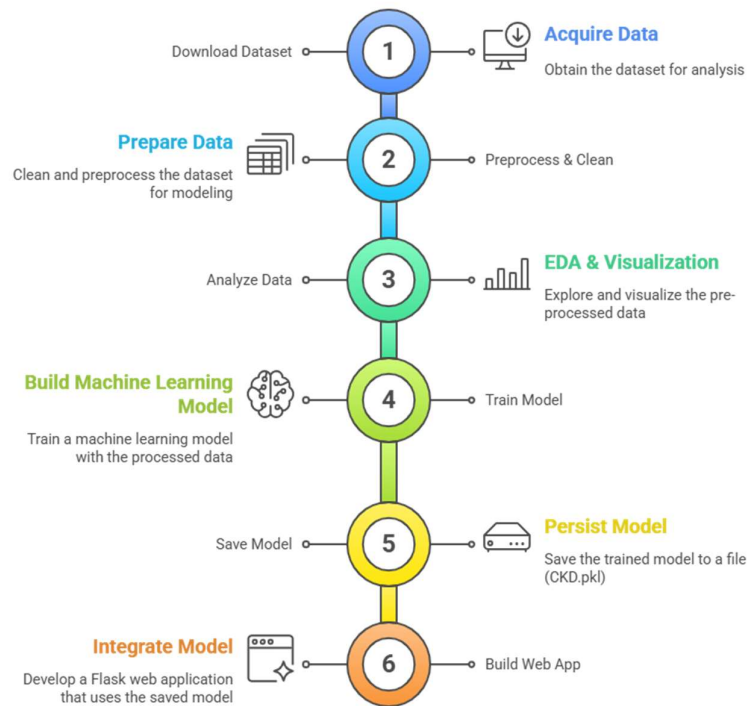
The models were evaluated using metrics like **accuracy, confusion matrix, and classification report.**

5. FLOW CHART

The following is the high-level workflow of the predictive system:

The process begins with **Data Acquisition and Cleaning**, where the raw clinical data (CSV format) is collected, preprocessed to handle missing values (using mean/mode imputation) and inconsistent entries, and categorical variables are encoded. Next is **Model Training and Selection**, involving splitting the data into training and testing sets and training various classification algorithms like Logistic Regression, Random Forest, and Naïve Bayes, ultimately selecting the model with the highest performance, which was the **Logistic Regression** model, reportedly achieving 97% accuracy. This is followed by **Model Saving and Integration**, where the optimal model is serialized (saved as CKD.pkl using pickle) and integrated as the predictive core within a web application built using the **Flask framework**. The final step is **User Prediction and Output**, where a user inputs their clinical values via the UI, the integrated model generates a binary prediction (CKD or Not CKD), and the result is displayed instantly.

Machine Learning Workflow for CKD Prediction



6. RESULT

Upon experimental investigation, the **Logistic Regression** model provided the highest performance among the tested algorithms.

The model chosen for deployment resulted in a high predictive capability:

The trained model was saved (e.g., 'CKD.pkl') and integrated into a web-based user interface (UI) to allow real-time prediction based on user-inputted lab values. The UI successfully displays a clear diagnosis (e.g., "Congratulations! You DON'T have Chronic Kidney Disease" or "Sorry! You have CHRONIC KIDNEY DISEASE. Please Consult Doctor.").



Predict Chronic Kidney Diseases Before It's Critical

Next-generation AI technology for early detection and prevention of Chronic Kidney Disease. Your health, reimagined.

Start Analysis

Learn More

UNDERSTANDING CKD

Comprehensive knowledge for better health decisions

Next-generation AI technology for early detection and prevention of Chronic Kidney Disease. Your health, reimagined.



What is CKD?

A progressive condition where kidneys lose filtering ability over time. Our AI detects early signs before symptoms appear.



Risk Factors

Diabetes, hypertension, obesity, smoking, and genetics. Our system analyzes 50+ parameters to assess your risk profile.



Early Symptoms

Fatigue, swelling, urination changes, nausea. ML algorithms detect subtle patterns in your health data.



Prevention

Stay hydrated, maintain a balanced diet, exercise regularly, and monitor blood sugar and pressure levels.



AI Insights

Our AI provides personalized health analytics and risk prediction to guide proactive healthcare decisions.



Track Progress

Monitor kidney function trends over time through intelligent dashboards and predictive health metrics.



Treatment

Options include medication, dialysis, and transplant. Early intervention is key to managing the condition effectively.



Lifestyle Mgmt

Manage CKD with a renal-friendly diet, stress reduction, and close cooperation with your healthcare team.

KEY RISK FACTORS

Understanding your risk factors is the first step toward prevention and early detection.



Diabetes

High blood sugar levels can damage kidney blood vessels.



High Blood Pressure

Hypertension is a leading cause of kidney damage.



Family History

Genetic predisposition increases CKD risk.



Age Factor

Risk increases with age, especially after 60.



Smoking

Damages blood vessels, reducing blood flow to the kidneys.



Obesity

Increases the risk of diabetes and high blood pressure.



Certain Medications

Long-term use of some drugs can harm kidney function.



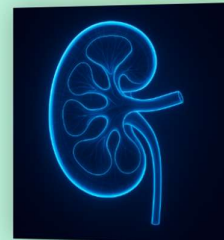
Ethnicity

Certain ethnic groups have a higher predisposition.



Chronic Kidney Disease (CKD)

Chronic Kidney Disease (CKD) is a long-term condition where the kidneys gradually lose their ability to filter waste and excess fluids from the blood. Understanding CKD is essential for early diagnosis, prevention, and effective treatment.



What is Chronic Kidney Disease?

Chronic Kidney Disease (CKD) refers to a progressive loss of kidney function over time. Kidneys play a vital role in filtering waste products, balancing electrolytes, and maintaining blood pressure. CKD develops when this function declines gradually, often due to conditions such as diabetes or hypertension.

CKD is classified into five stages based on the glomerular filtration rate (GFR), ranging from mild (Stage 1) to complete kidney failure (Stage 5).

Symptoms of CKD

In the early stages, CKD often shows no symptoms. However, as it progresses, patients may experience:

- Fatigue and weakness
- Swelling in the ankles, feet, or hands (edema)
- Shortness of breath
- Frequent urination (especially at night)
- Loss of appetite or nausea
- Muscle cramps or itching
- High blood pressure and irregular heartbeat

Causes of CKD

Several factors can contribute to the development of CKD, including:

- Diabetes mellitus (Type 1 and Type 2)
- Hypertension (high blood pressure)
- Chronic glomerulonephritis or kidney inflammation
- Polycystic kidney disease (genetic disorder)
- Prolonged use of certain medications (e.g., painkillers, NSAIDs)
- Urinary tract blockages or recurrent infections

Precautions & Prevention

Although CKD cannot always be prevented, the following precautions can slow its progression and reduce risks:

- Maintain a healthy diet low in salt, sugar, and saturated fats.
- Monitor blood pressure and blood sugar regularly.
- Stay hydrated and avoid excessive alcohol or tobacco use.
- Exercise regularly to maintain body weight and cardiovascular health.
- Avoid unnecessary use of painkillers or nephrotoxic drugs.
- Schedule periodic kidney function tests if you are at risk.

Impact of CKD

Untreated CKD can lead to severe complications, including:

- Fluid retention leading to swelling and high blood pressure
- Anemia and bone mineral disorders
- Electrolyte imbalances causing irregular heartbeat
- Cardiovascular diseases and stroke
- End-stage renal disease (ESRD) requiring dialysis or transplant

CKD Risk Assessment

Please fill out the fields below for a preliminary analysis.

Please enter your detail as per your test report

 Enter Your Blood Urea Level

e.g., 55

 Enter your Blood Glucose Random Level

e.g., 140 (the top number)

 Do you have deficiency of Anemia

No

 Do you have Coronary Artery Disease

No

 Please select your pus cell status

Normal

 Please Select your RBC status (as per your test report)

Normal

 Do you have Diabetes Mellitus

No

 Do you have Pedal Edema (swelling in the feet or ankles)

No

Analyze My Risk

PREVENTION PROTOCOL

Evidence-based strategies for kidney health



Hydration Optimization

Maintain 2-3 liters daily water intake. Proper hydration helps kidneys flush toxins and maintain optimal function.



Nutritional Balance

Low sodium, controlled protein, high fiber diet. Reduce processed foods and focus on whole grains, fruits, vegetables.



Regular Exercise

Engage in at least 30 minutes of moderate activity most days. Exercise improves circulation and overall kidney health.



Blood Pressure Control

Keep blood pressure within healthy limits through diet, exercise, and medication if needed. High BP damages kidneys.



Blood Sugar Management

Crucial for diabetics to prevent kidney damage. Monitor levels closely and adhere to treatment plans.



Avoid Smoking

Smoking severely impacts kidney function and accelerates damage. Quitting is a vital step for kidney health.



Medication Awareness

Be cautious with NSAIDs and other nephrotoxic drugs. Consult your doctor about kidney-safe alternatives.



Regular Check-ups

Annual physicals with kidney function tests are important, especially if you have risk factors.



EDUCATIONAL LIBRARY

Expert insights and patient stories



Understanding CKD

Comprehensive guide covering causes, stages, and progression of chronic kidney disease.



Kidney-Friendly Nutrition

Dietary strategies and meal plans optimized for kidney health and disease management.



Living with Dialysis

Patient stories and expert advice on managing life while on dialysis treatment.

WHY EARLY DETECTION MATTERS

Our AI-powered prediction system helps identify CKD before symptoms appear, giving you the opportunity to take preventive action.

- ✓ Early detection saves lives
- ✓ Prevent progression to kidney failure
- ✓ Reduce healthcare costs
- ✓ Improve quality of life
- ✓ Personalized treatment plans
- ✓ Peace of mind through screening

Ready to Check Your Risk?

Take our comprehensive risk assessment and get personalized recommendations for maintaining optimal kidney health.

[Start Your Assessment](#)

Free • Confidential • Takes 5 minutes

Get In Touch

We're here to help. Whether you have a question about our project, need support, or want to collaborate, please reach out to us.



Contact Information & Form

You can reach us directly through the following channels. We look forward to hearing from you.

Address:
123 Health Tech Avenue, Bengaluru, KA 560001, India

Email:
info@predictckd.com

Phone:
+91 987 654 3210

Full Name

Email Address

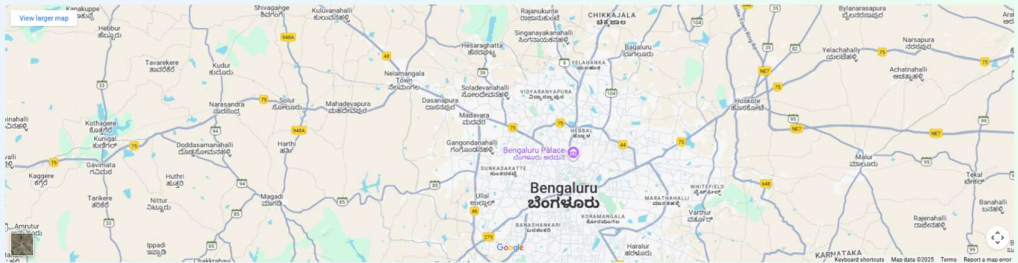
Subject

Message

[Send Message](#)

Our Location

Find us at our office in Bengaluru.



KidneyClear AI

Our AI-powered platform provides early-stage chronic kidney disease detection and personalized risk management.



Our Platform

How It Works
For Patients
For Healthcare Providers
Our Technology

Resources

What is CKD?
Prevention Protocol
Educational Library
Blog & News
Patient Stories

Company

About Us
Press & Media
Careers
Contact Us

Legal

Privacy Policy
Terms of Use
HIPAA Compliance
Accessibility

© 2025 KidneyClear AI. All rights reserved. This information is for educational purposes and not a substitute for professional medical advice.

Made by BKP SA SST & UK

Chronic Kidney Disease

A Machine Learning Web App, Built with Flask



High Risk Detected

Our analysis indicates a high probability of Chronic Kidney Disease based on the provided data. Please consult a healthcare professional for a complete diagnosis and guidance.

Model Confidence: 99.84%

← Run New Assessment

Chronic Kidney Disease

A Machine Learning Web App, Built with Flask



Low Risk Detected

Our analysis indicates a low probability of Chronic Kidney Disease based on the provided data. We still recommend regular check-ups and maintaining a healthy lifestyle.

Model Confidence: 73.4%

← Run New Assessment

7. ADVANTAGES / DISADVANTAGES

ADVANTAGES

- **Ease of Prediction:** The result can be predicted quickly based on test values without the immediate help of a doctor.
- **Early Intervention:** Helps patients and physicians to take necessary measures to cure CKD before complete kidney failure.
- **Efficiency:** Increases the speed of working with large datasets, generating rapid reports, and improving operational efficiency.
- **Predictive Knowledge:** Data mining and ML can extract predictive knowledge from large databases.

DISADVANTAGES

- **Accuracy Fluctuation:** The model sometimes may not give entirely accurate results, leading to wrong predictions.
- **Data Concerns:** Involves privacy and security issues related to human data administration and potential misuse of incorrect information.

8. APPLICATIONS

This predictive model has key applications in the healthcare sector:

- **Clinical Screening:** Used as a preliminary screening tool to predict CKD status based on clinical data.
- **Websites/Apps:** Easily deployed as a website or a mobile application for general public use, making early detection more accessible.
- **Risk Assessment:** Useful for predicting kidney diseases in the present situation, especially for patients with risk factors like hypertension and diabetes.

9. CONCLUSION

This project successfully demonstrates the use of machine learning to predict Chronic Kidney Disease status. By rigorously preprocessing a clinical dataset and applying classification algorithms, a robust predictive model was created. The **Logistic Regression** model offers a highly accurate tool that allows users to easily predict their disease status quickly, supporting the essential need for early detection and intervention in CKD management.

10. FUTURE SCOPE

The future scope for this model is very high, given the global shift towards machine learning in medicine. Future work should focus on:

- Integrating the model with **Electronic Health Records (EHR)** systems for automated, real-time risk assessment.
- Exploring and implementing **Deep Learning** architectures for potentially higher predictive performance.
- Expanding the dataset with longitudinal patient data to predict the *progression* of CKD, not just the current presence.
- Improving the user interface and potentially building a mobile application version of the predictor.

11. BIBLIOGRAPHY

- Flask Basics : https://www.youtube.com/watch?v=lj4l_CvBnt0
- Installation : <https://youtu.be/5mDYijMfSzs>
- Packages Installation : https://youtu.be/akj3_wTploU
- Supervised and Unsupervised Learning : https://youtu.be/kE5QZ8G_78c
- Regression classification and clustering : https://youtu.be/6za9_mh3uTE
- Logistic Regression : <https://youtu.be/yIYKR4sgzl8>

12. APPENDIX

➤ **Installations**

```
!pip install pandas  
!pip install seaborn
```

➤ **Data Pre-processing**

```
import pandas as pd  
import numpy as np  
import seaborn as sns  
import matplotlib.pyplot as plt  
from sklearn.metrics import confusion_matrix, accuracy_score  
from sklearn.preprocessing import LabelEncoder  
from sklearn.model_selection import train_test_split
```

```
data = pd.read_csv("Dataset/kidney_disease.csv")
```

```
data.head() # it will print top 5 rows
```

```
data.tail() # it will print last 5
```

```
data.head(20) # it will print first 20 rows
```

```
data.tail(20) # it will print last 20 rows
```

```
data.drop(["id"], axis = 1, inplace = True) # it will drop(remove) id column
```

```
data.info() # it gives information of data mean data type
```

```
data.describe() # it computes summary values for continuous column data mean std,mean,count etc
```

```
data['classification'].unique() #return unique value of classification column
```

```
data['classification']=data['classification'].replace("ckd","ckd\t") #replace ckt by ckd\t
data['classification'].unique()
```

```
catcols = ['rbc', 'pc', 'pcc', 'ba', 'htn', 'dm', 'cad', 'appet', 'pe', 'ane', 'classification']
print(catcols)
```

```
from collections import Counter as c
for i in catcols:
    print("columns:",i)
    print(c(data[i])) # using counter for checking number of classes in cloumn
    print('*'*120 + '\n')
```

```
for col in ['rbc', 'rc', 'wc', 'pcv']:
    if col in catcols:
        catcols.remove(col)
```

```
cont_cols = ['age', 'bp', 'sg', 'al', 'su', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hemo', 'pcv', 'wc', 'rc']
print(cont_cols)
```

```
from collections import Counter as c
for i in cont_cols:
    print("Continuos columns:",i)
    print(c(data[i])) # using counter for checking number of classes in cloumn
    print('*'*120 + '\n')
```

```
cont_cols.remove('su')
cont_cols.remove('bp')
cont_cols.remove('sg')
print(cont_cols)
```

```
cont_cols.append('rc')
cont_cols.append('pcv')
cont_cols.append('wc')
cont_cols.append('rbc')
print(cont_cols)
```

```
# catcols.add('rbc') # rbc should be numerical
print(catcols)
```

```
print(cont_cols)
```

➤ Rectifying the Categorical Column Classes

```
data['cad'] = data.cad.replace('\tno','no')
```

```
c(data['cad'])
```

```
data['dm'] = data.dm.replace('\tno','no')
c(data['dm'])
```

➤ Null Value Checking

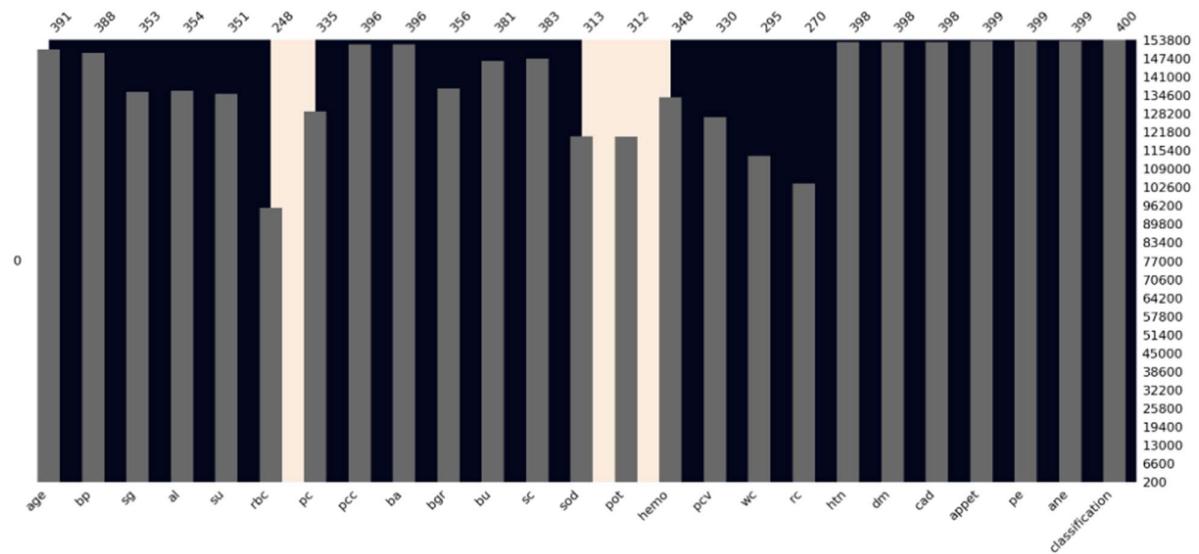
```
data.isnull().any()
```

```
data.isnull().sum()
```

```
import missingno as msno
```

```
sns.heatmap(data.isnull(),cbar=False)
```

```
msno.bar(data)
plt.show()
```



```
data.pcv = pd.to_numeric(data.pcv, errors='coerce')
data.wc = pd.to_numeric(data.wc, errors='coerce')
```

➤ Handling Continuous/numerical Column Null Values

```
data['bgr'].fillna(data['bgr'].mean(),inplace=True)
data['bp'].fillna(data['bp'].mean(),inplace=True)
data['bu'].fillna(data['bu'].mean(),inplace=True)
data['hemo'].fillna(data['hemo'].mean(),inplace=True)
data['pcv'].fillna(data['pcv'].mean(),inplace=True)
data['pot'].fillna(data['pot'].mean(),inplace=True)
# data['rbc'].fillna(data['rbc'].mean(),inplace=True)
```



```
data['sc'].fillna(data['sc'].mean(),inplace=True)
data['sod'].fillna(data['sod'].mean(),inplace=True)
data['wc'].fillna(data['wc'].mean(),inplace=True)
```

```
for col in catcols:
    if data[col].dtype == 'object':
        # Replace incorrect values and strip whitespace
        data[col] = data[col].str.strip()
        data[col] = data[col].replace(['\t?', '?', '\tno', '\tyes', 'yes'], [np.nan, np.nan, 'no', 'yes', 'yes'])

        # Manually find the mode based on non-null value counts
        value_counts = data[col].value_counts()
        if not value_counts.empty:
            mode_value = value_counts.idxmax()
            data[col].fillna(mode_value, inplace=True)
            print(f"Mode for column {col} (manual): {mode_value}")
        else:
            data[col].fillna('unknown', inplace=True)
            print(f"Filled NaNs in {col} with 'unknown' as no valid values found after cleaning.")
```

```
for col in cont_cols:
    if data[col].dtype != 'object':
        data[col].fillna(data[col].mean(),inplace=True)
    else: # Handle numerical columns that were read as objects
        data[col] = pd.to_numeric(data[col], errors='coerce')
        data[col].fillna(data[col].mean(),inplace=True)
```

```
data.isnull().sum()
```

```
catcols = ['ane', 'pe', 'appet', 'cad', 'dm', 'htn', 'ba', 'pcc', 'pc', 'rbc']
```

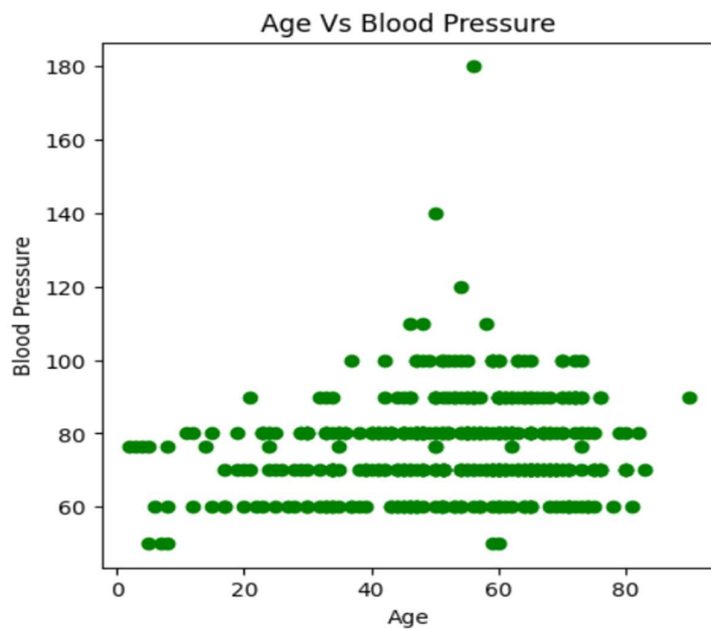
```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
for i in catcols:
    print("Label Encoding of :", i)
    print(c(data[i]))
    data[i] = le.fit_transform(data[i])
    print(c(data[i]))
    print(""*100)
```

➤ Exploratory Analysis

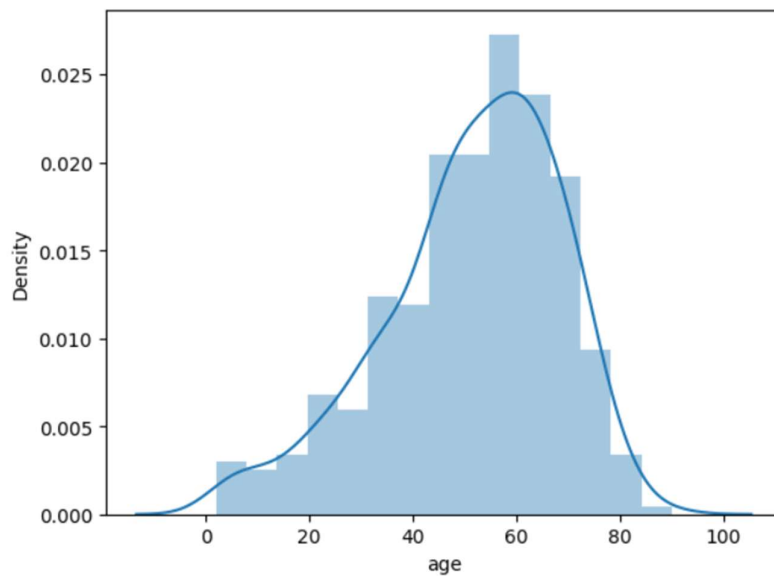
```
data.describe()
```

➤ Data Visualization

```
#Age Vs Blood Pressure
fig=plt.figure(figsize=(5,5))
plt.scatter(data['age'],data['bp'],color='green')
plt.xlabel('Age')
plt.ylabel('Blood Pressure')
plt.title('Age Vs Blood Pressure')
plt.show()
```



```
sns.distplot(data.age)
```

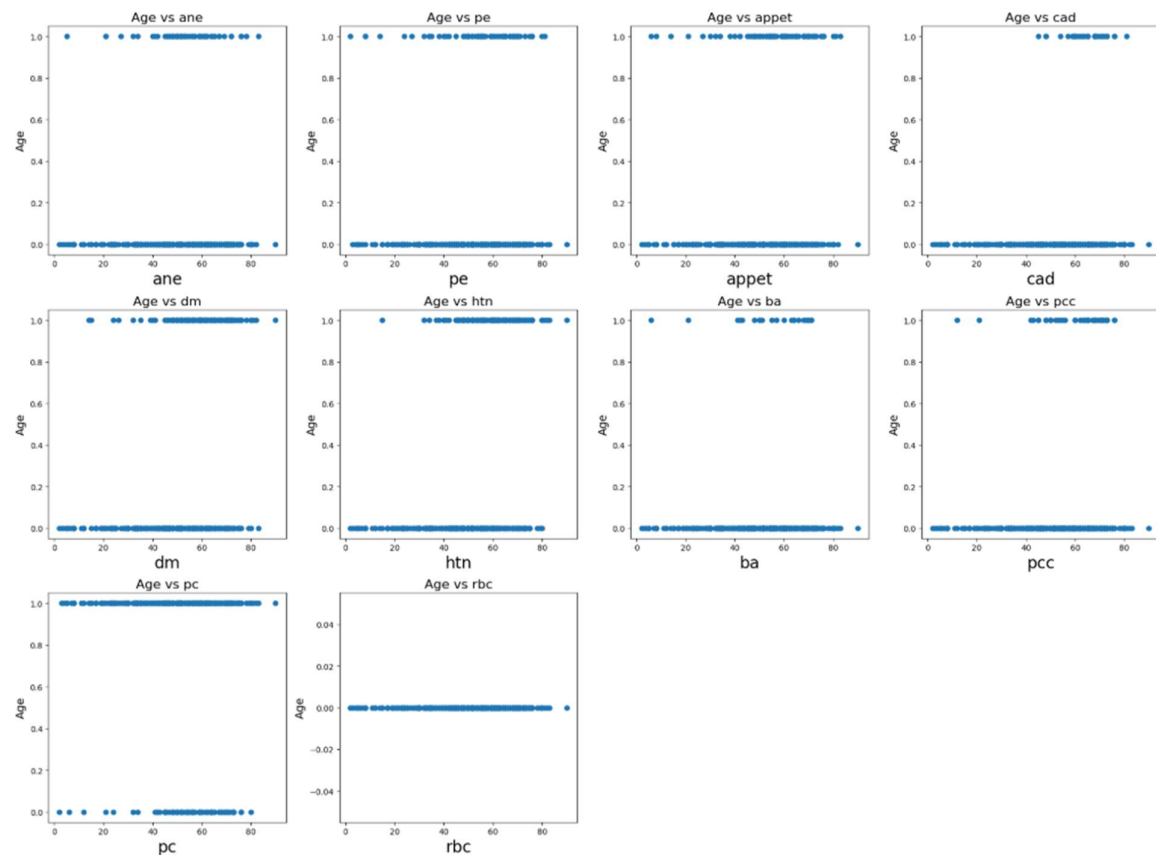


```
for column in catcols:
    data[column] = pd.Categorical(data[column]).codes
plt.figure(figsize=(20,15),facecolor='white')
```

```

plotnumber=1
for column in catcols:
    if plotnumber<=11:
        ax = plt.subplot(3,4,plotnumber) # 3,4 is refer to 3X4 matrix
        plt.scatter(data['age'],data[column]) #plotting scatter plot
        plt.xlabel(column,fontsize=20)
        plt.ylabel('Age', fontsize=14)
        plt.title(f'Age vs {column}', fontsize=16)
        plotnumber+=1
plt.tight_layout()
plt.show()

```

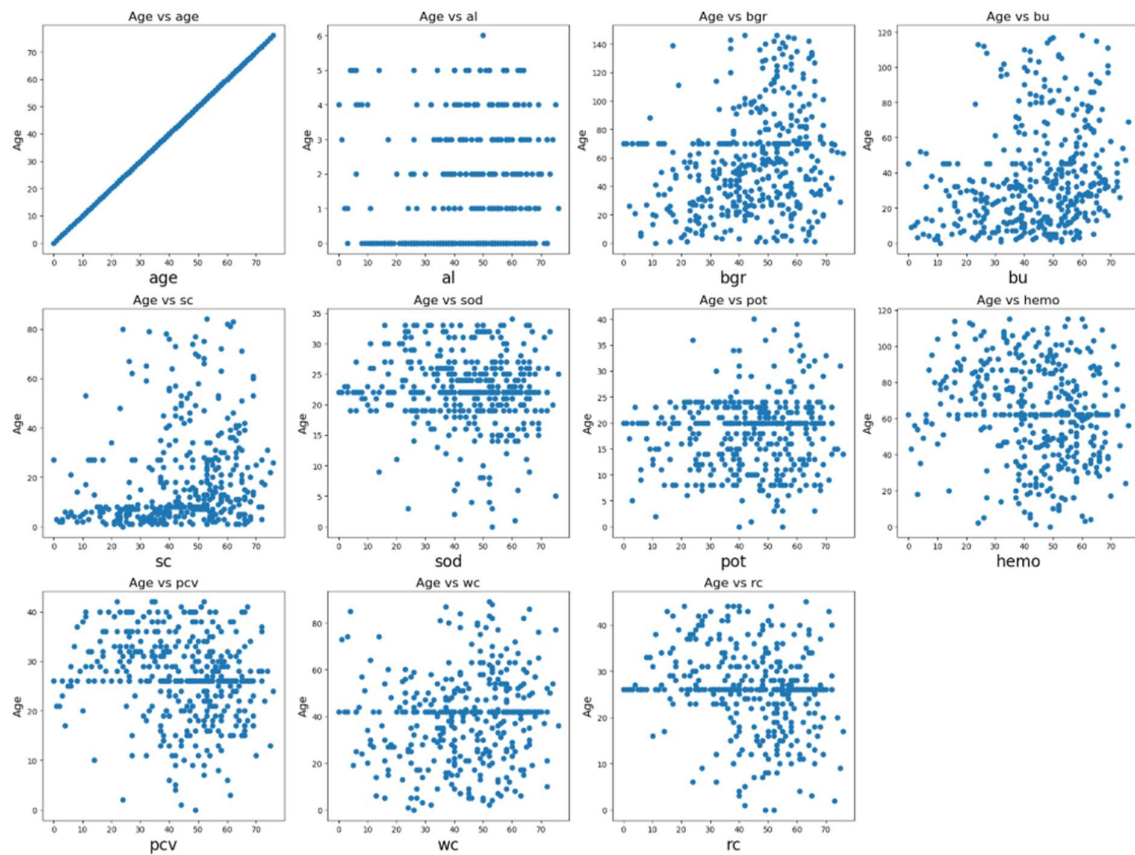


```

for column in cont_cols:
    data[column] = pd.Categorical(data[column]).codes
plt.figure(figsize=(20,15),facecolor='white')
plotnumber=1
for column in cont_cols:
    if plotnumber<=11:
        ax = plt.subplot(3,4,plotnumber) # 3,4 is refer to 3X4 matrix
        plt.scatter(data['age'],data[column]) #plotting scatter plot
        plt.xlabel(column,fontsize=20)
        plt.ylabel('Age', fontsize=14)
        plt.title(f'Age vs {column}', fontsize=16)
        plotnumber+=1

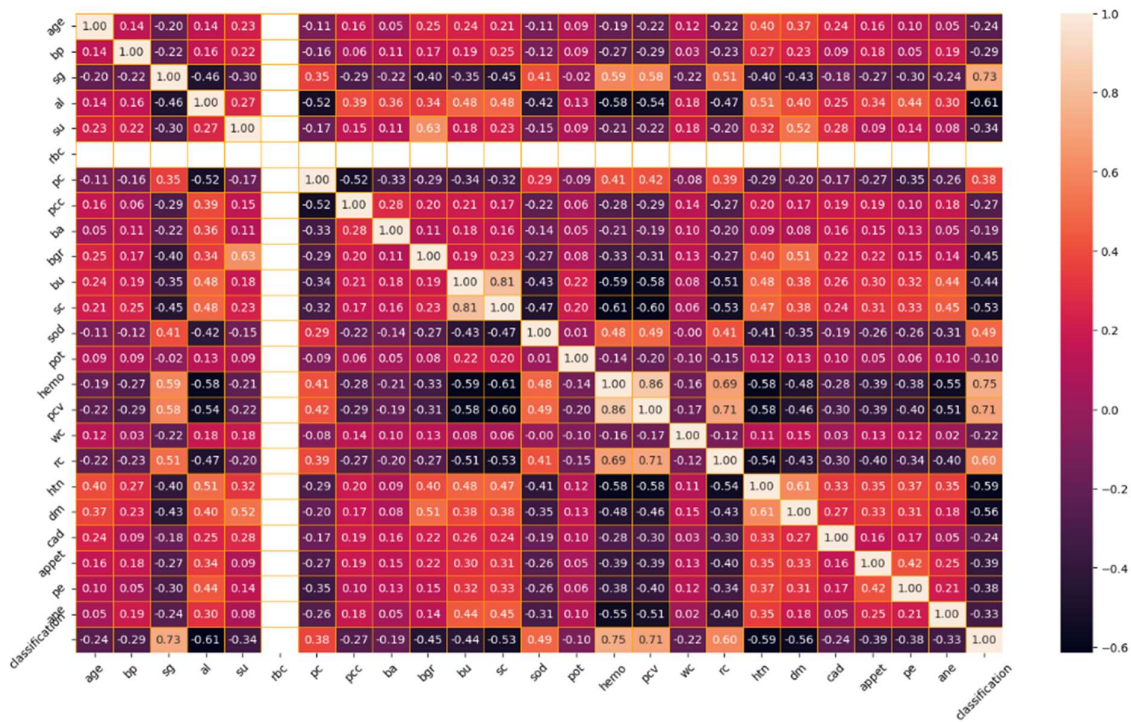
```

```
plt.tight_layout()
plt.show()
```

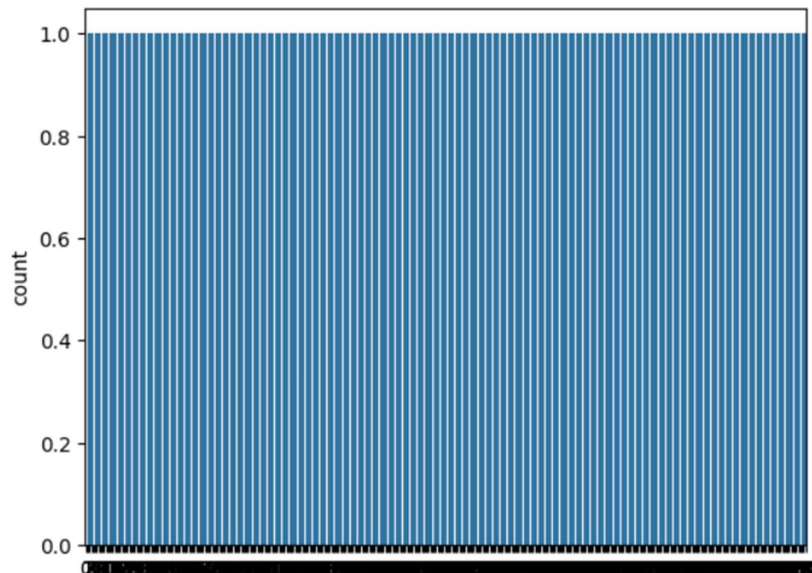


```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
data['classification'] = le.fit_transform(data['classification'])
```

```
#HEAT MAP #correlation of parameters
f,ax=plt.subplots(figsize=(18,10))
sns.heatmap(data.corr(),annot=True,fmt=".2f",ax=ax,linewidths=0.5,linestyle="orange")
plt.xticks(rotation=45)
plt.yticks(rotation=45)
plt.show()
```



```
sns.countplot(data['classification'])
```



```
# performing feature Scaling operation using standard scaller on X part of the dataset because
# there different type of values in the columns
selcols=['rbc','pc','bgr','bu',
        'pe','ane','dm','cad']
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
#x = data.drop(columns=['classification'])
x=pd.DataFrame(data,columns=selcols)
x_bal=sc.fit_transform(x)
```

```

x=pd.DataFrame(data,columns=selcols)
y=pd.DataFrame(data,columns=['classification'])

print(x.shape)
print(y.shape)

# Splitting the data into train and test

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=2) #train test split

print("\nShape of x_train:", x_train.shape)
print("Shape of x_test:", x_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

```

➤ **Training The Model in Multiple Algorithm**

```

#ANN Model
# Importing the Keras libraries and packages
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Creating ANN skleton view
classification = Sequential()
classification.add(Dense(30,activation='relu'))
classification.add(Dense(128,activation='relu'))
classification.add(Dense(64,activation='relu'))
classification.add(Dense(32,activation='relu'))
classification.add(Dense(1,activation='sigmoid'))
# Compiling the ANN model
classification.compile(optimizer='adam',loss='binary_crossentropy',metrics=['accuracy'])

# Training the model
# Storing the training history
history = classification.fit(x_train,y_train,batch_size=10,validation_split=0.2,epochs=500)

#Random forest Model
from sklearn.ensemble import RandomForestClassifier
rfc = RandomForestClassifier(n_estimators=10,criterion='entropy')

# Fit the model
# We use .ravel() on y_train to fix the DataConversionWarning
rfc.fit(x_train,y_train.ravel())

```

▼ RandomForestClassifier ⓘ ?

► Parameters

```
# Make predictions
y_predict = rfc.predict(x_test)
y_predict_train = rfc.predict(x_train)
print("\nRandom Forest Test Predictions:", y_predict)
```

```
print("Random Forest Train Predictions:", y_predict_train)
```

```
#Decision Tree Model
from sklearn.tree import DecisionTreeClassifier
dtc = DecisionTreeClassifier(max_depth=4,splitter='best',criterion='entropy')
```

```
# Fit the model
# Using .ravel() on y_train to prevent DataConversionWarning
dtc.fit(x_train,y_train)
```

```
# Make predictions
y_predict = dtc.predict(x_test)
y_predict_train = dtc.predict(x_train)
```

```
print("\nDecision Tree Test Predictions:", y_predict)
```

```
print("Decision Tree Train Predictions:", y_predict_train)
```

```
#Logistic Regression
from sklearn.linear_model import LogisticRegression
lgr = LogisticRegression()
lgr.fit(x_train,y_train.values.ravel()) # Use .values.ravel() to prevent DataConversionWarning
```

▼ LogisticRegression ⓘ ?

► Parameters

```
from sklearn.metrics import accuracy_score,classification_report
y_predict = lgr.predict(x_test)
print("\nLogistic Regression Test Predictions:", y_predict)
```

➤ **Testing the Model**

```
from sklearn.metrics import accuracy_score, classification_report
# logistic Regression
y_pred = lgr.predict(x_test)
print("\n# logistic Regression")
print(y_pred)
print(x_test)
```

```
y_pred = dtc.predict(x_test)
print("\n# DecisionTree Classifier")
print(y_pred)
```

```
y_pred = rfc.predict(x_test)
print("\n# Random Forest Classifier")
print(y_pred)
```

```
classification.save("ckd.keras")
```

```
y_pred=classification.predict(x_test)
```

```
y_pred
```

```
y_pred=(y_pred>0.5)
y_pred
```

```
def predict_exit(sample_value):
    sample_value=np.array(sample_value)
    sample_value=sample_value.reshape(1,-1)
    sample_value=sc.transform(sample_value)
    return classifier.predict(sample_value)
```

```
import numpy as np
```

```
new_data_point = np.array([[1,1,121.00000,36.0,0.0,0,1,0]]) # Using values from image
test=classification.predict(new_data_point)
if test > 0.5: # Use > 0.5 for sigmoid output
    print('Prediction:High chance of CKD!')
else:
    print('Prediction:Low chance of CKD')
```


➤ **Testing model with multiple evaluation metrics**

```
#Compare Model
from sklearn import model_selection

dfs = [] # Initialize dfs as an empty list
models = [
    ('LogReg', LogisticRegression(max_iter=1000)), # Increased max_iter
    ('RF', RandomForestClassifier()),
    ('DecisionTree', DecisionTreeClassifier())
]

results = []
names = []
scoring = ['accuracy', 'precision_weighted', 'recall_weighted', 'f1_weighted', 'roc_auc']
target_names = ['NO CKD', 'CKD']
for name, model in models:
    kfold = model_selection.KFold(n_splits=5, shuffle=True, random_state=90210)
    cv_results = model_selection.cross_validate(model, x_train, y_train.values.ravel(), cv=kfold,
scoring=scoring) # Use .values.ravel()
    clf = model.fit(x_train, y_train.values.ravel()) # Use .values.ravel()
    y_pred = clf.predict(x_test)
    print(name)
    print(classification_report(y_test, y_pred, target_names=target_names))
    results.append(cv_results)
    names.append(name)
    this_df = pd.DataFrame(cv_results)
    this_df['model'] = name
    dfs.append(this_df)

final = pd.concat(dfs, ignore_index=True)
# return final # Removed return statement
```

LogReg					
	precision	recall	f1-score	support	
NO CKD	1.00	0.89	0.94	54	
CKD	0.81	1.00	0.90	26	
accuracy			0.93	80	
macro avg	0.91	0.94	0.92	80	
weighted avg	0.94	0.93	0.93	80	
RF					
	precision	recall	f1-score	support	
NO CKD	0.96	0.96	0.96	54	
CKD	0.92	0.92	0.92	26	
accuracy			0.95	80	
macro avg	0.94	0.94	0.94	80	
weighted avg	0.95	0.95	0.95	80	
DecisionTree					
	precision	recall	f1-score	support	
NO CKD	0.91	0.94	0.93	54	
CKD	0.88	0.81	0.84	26	
...					
accuracy			0.90	80	
macro avg	0.89	0.88	0.88	80	
weighted avg	0.90	0.90	0.90	80	

```

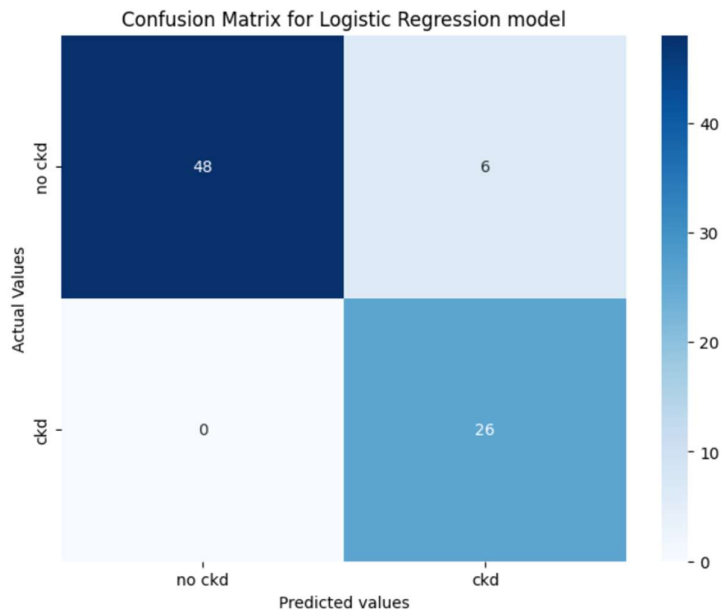
from sklearn.metrics import confusion_matrix
# Calculate predictions for Logistic Regression
y_predict_lr = lgr.predict(x_test)
cm_lr = confusion_matrix(y_test, y_predict_lr)
cm_lr

```

```

# Plotting confusion matrix for Logistic Regression
plt.figure(figsize=(8,6))
sns.heatmap(cm_lr, cmap='Blues', annot=True, xticklabels=['no ckd', 'ckd'], yticklabels=['no ckd', 'ckd'])
plt.xlabel('Predicted values')
plt.ylabel('Actual Values')
plt.title('Confusion Matrix for Logistic Regression model')
plt.show()

```



Making the Confusion Matrix for RandomForestClassifier

from sklearn.metrics import confusion_matrix

Calculate predictions for Random Forest

y_predict_rf = rfc.predict(x_test)

cm_rf = confusion_matrix(y_test, y_predict_rf)

cm_rf

print(y_test)

print(y_predict_rf)

Plotting confusion matrix for RandomForestClassifier

plt.figure(figsize=(8,6))

sns.heatmap(cm_rf, cmap='Blues', annot=True, xticklabels=['no ckd', 'ckd'], yticklabels=['no ckd', 'ckd'])

print(y_test)

print(x_test)

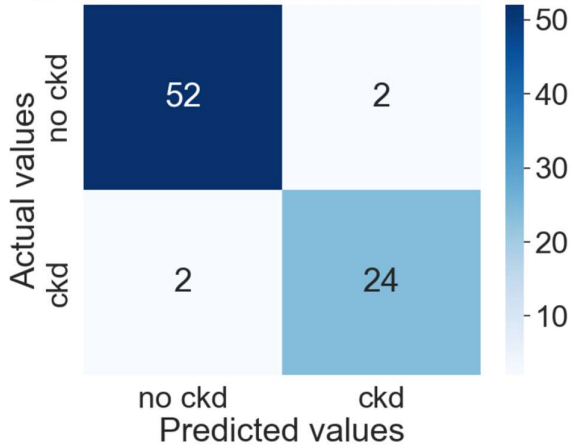
plt.xlabel('Predicted values')

plt.ylabel('Actual values')

plt.title('Confusion Matrix for RandomForestClassifier')

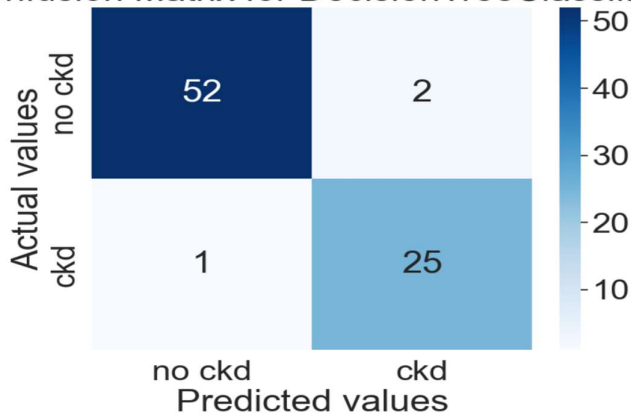
plt.show()

Confusion Matrix for RandomForestClassifier



```
# Plotting confusion matrix for DecisionTreeClassifier
# Calculate predictions for Decision Tree
y_predict_dtc = dtc.predict(x_test)
cm_dtc = confusion_matrix(y_test, y_predict_dtc)
print(y_test)
print(x_test)
plt.figure(figsize=(8,6))
sns.heatmap(cm_dtc, cmap='Blues', annot=True, xticklabels=['no ckd', 'ckd'], yticklabels=['no ckd', 'ckd'])
plt.xlabel('Predicted values')
plt.ylabel('Actual values')
plt.title('Confusion Matrix for DecisionTreeClassifier')
plt.show()
```

Confusion Matrix for DecisionTreeClassifier

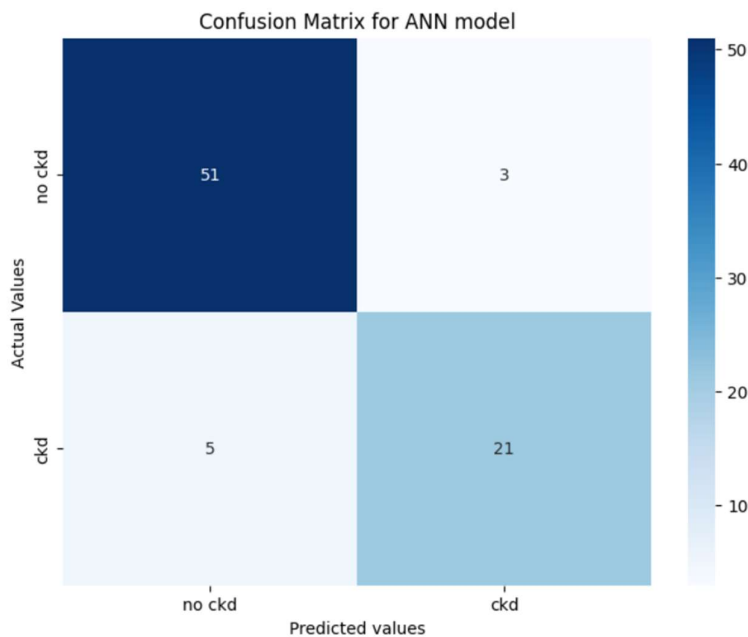


```
# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(y_test)
print(x_test)
cm
```

```

# Plotting confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, cmap='Blues', annot=True, xticklabels=['no ckd', 'ckd'], yticklabels=['no ckd', 'ckd'])
plt.xlabel('Predicted values')
plt.ylabel('Actual Values')
plt.title('Confusion Matrix for ANN model')
plt.show()

```



➤ Evaluate the results

```

bootstraps = []
for model in list(set(final.model.values)):
    model_df = final.loc[final.model == model]
    bootstrap = model_df.sample(n=30, replace=True)
    bootstraps.append(bootstrap)

bootstrap_df = pd.concat(bootstraps, ignore_index=True)
results_long = pd.melt(bootstrap_df, id_vars=['model'], var_name='metrics', value_name='values')
time_metrics = ['fit_time', 'score_time'] # fit time metrics
## PERFORMANCE METRICS
results_long_nofit = results_long.loc[~results_long['metrics'].isin(time_metrics)] # get df without fit data
results_long_nofit = results_long_nofit.sort_values(by='values')
## TIME METRICS
results_long_fit = results_long.loc[results_long['metrics'].isin(time_metrics)] # df with fit data
results_long_fit = results_long_fit.sort_values(by='values')

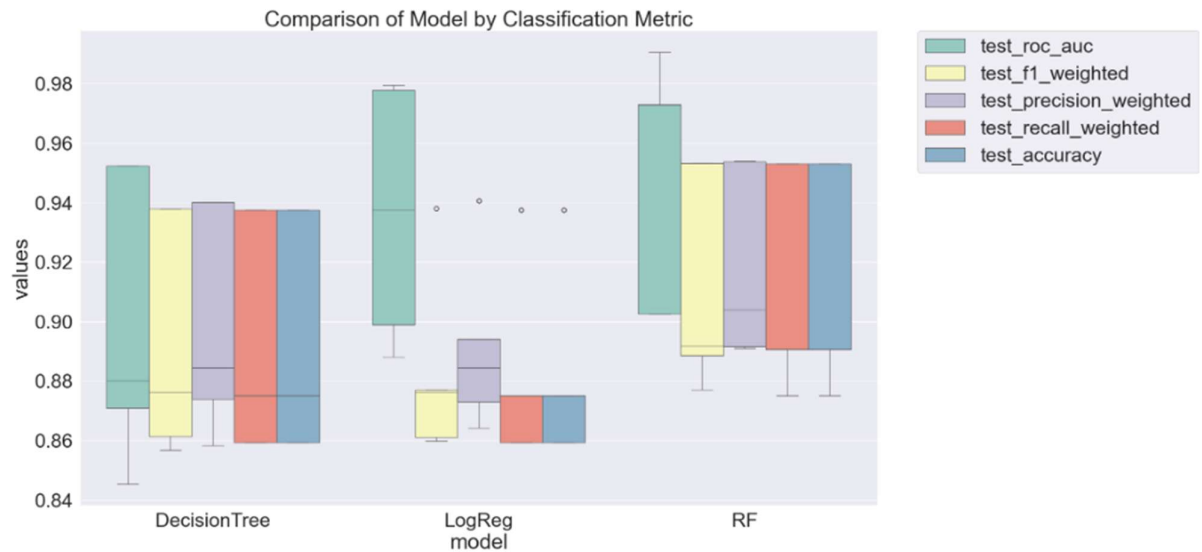
```

```
import matplotlib.pyplot as plt
```

```

import seaborn as sns
plt.figure(figsize=(20, 12))
sns.set(font_scale=2.5)
g = sns.boxplot(x="model", y="values", hue="metrics", data=results_long_nofit, palette="Set3")
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.title('Comparison of Model by Classification Metric')
plt.savefig('./benchmark_models_performance.png',dpi=300)

```



➤ Saving The Best Model

```

import pickle
pickle.dump(lgr,open('CKD.pkl','wb'))

```